

# LiDAR Obstacle Avoidance

## LiDAR Obstacle Avoidance

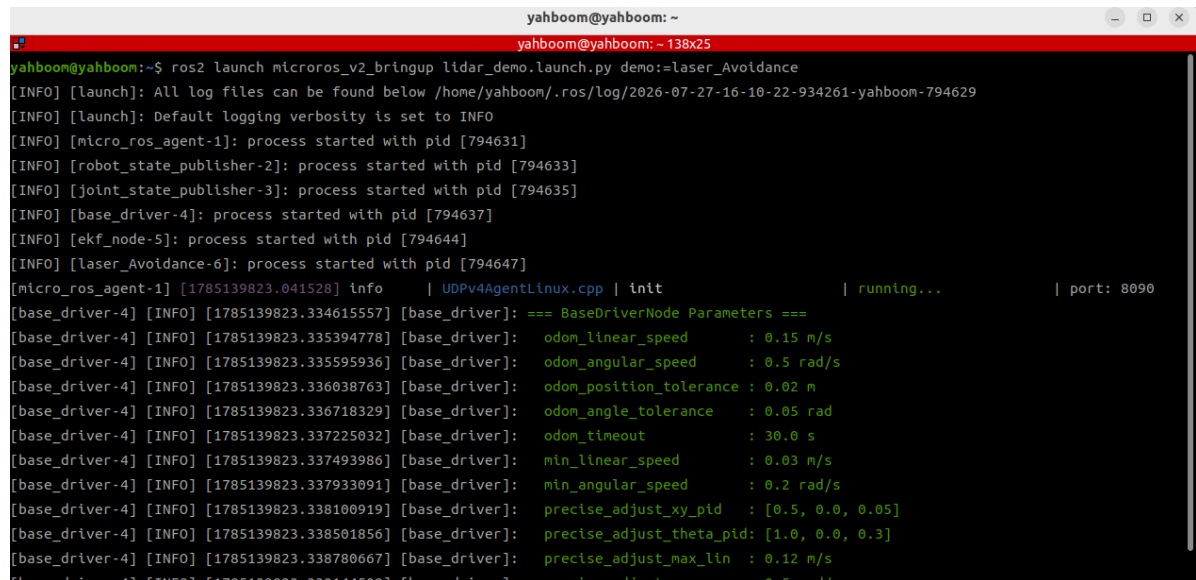
1. Course Content
2. Demo
3. Source Code Analysis

## 1. Course Content

- Start the demo program: the robot uses the LiDAR to detect obstacles ahead and automatically turns to avoid them.

## 2. Demo

```
ros2 launch microros_v2_bringup lidar_demo.launch.py demo:=laser_Avoidance
```



```
yahboom@yahboom: ~  
yahboom@yahboom: ~ 138x25  
yahboom@yahboom:~$ ros2 launch microros_v2_bringup lidar_demo.launch.py demo:=laser_Avoidance  
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-27-16-10-22-934261-yahboom-794629  
[INFO] [launch]: Default logging verbosity is set to INFO  
[INFO] [micro_ros_agent-1]: process started with pid [794631]  
[INFO] [robot_state_publisher-2]: process started with pid [794633]  
[INFO] [joint_state_publisher-3]: process started with pid [794635]  
[INFO] [base_driver-4]: process started with pid [794637]  
[INFO] [ekf_node-5]: process started with pid [794644]  
[INFO] [laser_Avoidance-6]: process started with pid [794647]  
[micro_ros_agent-1] [1785139823.041528] info | UDPv4AgentLinux.cpp | init | running... | port: 8090  
[base_driver-4] [INFO] [1785139823.334615557] [base_driver]: === BaseDriverNode Parameters ===  
[base_driver-4] [INFO] [1785139823.335394778] [base_driver]: odom_linear_speed : 0.15 m/s  
[base_driver-4] [INFO] [1785139823.335595936] [base_driver]: odom_angular_speed : 0.5 rad/s  
[base_driver-4] [INFO] [1785139823.336038763] [base_driver]: odom_position_tolerance : 0.02 m  
[base_driver-4] [INFO] [1785139823.336718329] [base_driver]: odom_angle_tolerance : 0.05 rad  
[base_driver-4] [INFO] [1785139823.337225032] [base_driver]: odom_timeout : 30.0 s  
[base_driver-4] [INFO] [1785139823.337493986] [base_driver]: min_linear_speed : 0.03 m/s  
[base_driver-4] [INFO] [1785139823.337933091] [base_driver]: min_angular_speed : 0.2 rad/s  
[base_driver-4] [INFO] [1785139823.338100919] [base_driver]: precise_adjust_xy_pid : [0.5, 0.0, 0.05]  
[base_driver-4] [INFO] [1785139823.338501856] [base_driver]: precise_adjust_theta_pid: [1.0, 0.0, 0.3]  
[base_driver-4] [INFO] [1785139823.338780667] [base_driver]: precise_adjust_max_lin : 0.12 m/s  
[base_driver-4] [INFO] [1785139823.339144580] [base_driver]: precise_adjust_max_ang : 0.5 rad/s
```

- While the robot moves forward, the LiDAR scans a 30° range in front in real time for obstacles
- When an obstacle is detected within 0.3 m ahead, the robot automatically turns to go around it
- After bypassing the obstacle, it resumes moving straight forward
- The detection range and the obstacle response distance can be tuned through the parameter file

## 3. Source Code Analysis

Source code path:

```
$HOME/microros_ws/src/common_demo/common_demo/lidar/laser_Avoidance.py
```

The core logic of the obstacle avoidance node is divided into two parts: **TF coordinate transform setup** and **laser point cloud processing + obstacle avoidance decision-making**.

1. **Wait for TF transform (wait\_for\_tf):** On startup it first waits 1 second so that `TransformListener` can cache TF data. If the `laser_frame` parameter is not specified, it waits for the `/scan` callback to auto-detect the LiDAR frame name. It then repeatedly queries the `base_link ← laser_frame` TF until it becomes available or the node exits, ensuring that the later point cloud transforms do not fail because TF is not ready.
2. **Laser callback (registerScan):**
  - **Point cloud traversal:** Each valid laser point (distance not 0 and less than `range_max`) is converted from polar coordinates to Cartesian coordinates `(x, y)`, then transformed to the `base_link` frame via `do_transform_point`, so that the bearing is judged correctly even when the LiDAR is mounted with an offset.
  - **Region division:** Based on the distance to `base_link` ( $\leq \text{ResponseDist} \times 1.5$ ) and the angle relative to the x axis, the points are classified into three regions:
    - `|angle| < LaserAngle` → front warning `front_warning`
    - `LaserAngle ≤ angle < 90°` → left warning `Left_warning`
    - `-90° < angle ≤ -LaserAngle` → right warning `Right_warning`
  - **Obstacle avoidance decision:**
    - Front clear → move straight forward, release the turn direction lock
    - Front blocked → rotate in place, **once a turn direction is chosen it is locked** until the front is clear again, avoiding left/right oscillation
    - Turn direction priority: left blocked → turn right; right blocked → turn left; both sides blocked → choose the side with fewer warning points

```
def wait_for_tf(self):
    """Wait for TF transform to be available"""
    self.get_logger().info('waiting for TF transform listener to be
ready...')

    # Wait for TransformListener to cache TF data
    time.sleep(1.0)

    # If laser_frame parameter is empty, wait for first scan data to detect
    actual frame_id
    if not self.laser_frame:
        self.get_logger().info('laser_frame parameter not set, will detect
from /scan topic')
        # wait a short time for sub_laser callback to receive data
        wait_count = 0
        while not self.laser_frame and wait_count < 10:
            time.sleep(0.5)
            wait_count += 1

        if not self.laser_frame:
            self.get_logger().error('Failed to detect laser frame from /scan
topic')

        return

    # Wait for TF transform to be available
    while rclpy.ok():
        try:
            self.tf_buffer.lookup_transform(
```

```

        'base_link',
        self.laser_frame,
        rclpy.time.Time(),
        timeout=rclpy.duration.Duration(seconds=1.0)
    )
    self.get_logger().info(f'TF available: base_link <-
{self.laser_frame}')
    self.tf_ready = True
    return
except TransformException:
    # Throttle TF waiting logs
    current_time = self.get_clock().now().nanoseconds / 1e9
    if (current_time - self.last_tf_log_time) >=
self.tf_log_interval:
        self.get_logger().info(f'waiting for TF: base_link <-
{self.laser_frame}...')
        self.last_tf_log_time = current_time
        time.sleep(0.5)

def registerScan(self, scan_data):
    if not isinstance(scan_data, LaserScan): return

    # Auto-detect laser frame_id
    if not self.laser_frame:
        self.laser_frame = scan_data.header.frame_id
        self.get_logger().info(f'Detected laser frame: {self.laser_frame}')

    # If TF is not ready, try to establish connection
    if not self.tf_ready:
        try:
            self.tf_buffer.lookup_transform(
                'base_link',
                scan_data.header.frame_id,
                rclpy.time.Time(),
                timeout=rclpy.duration.Duration(seconds=0.1)
            )
            self.tf_ready = True
            self.get_logger().info(f'TF ready: base_link <-
{scan_data.header.frame_id}')
        except TransformException:
            # TF not ready, skip this scan
            return

    # Reset counters
    self.Right_warning = 0
    self.Left_warning = 0
    self.front_warning = 0

    # Get transform from laser to base_link
    try:
        trans = self.tf_buffer.lookup_transform(
            'base_link',
            scan_data.header.frame_id,
            rclpy.time.Time(),
            timeout=rclpy.duration.Duration(seconds=0.1)
        )
    except TransformException:
        # Throttle TF error logs

```

```

        current_time = self.get_clock().now().nanoseconds / 1e9
        if (current_time - self.last_tf_log_time) >= self.tf_log_interval:
            self.get_logger().warn('TF not available, skipping scan')
            self.last_tf_log_time = current_time
        return

# Iterate through all laser points
ranges = np.array(scan_data.ranges)
for i in range(len(ranges)):
    if ranges[i] == 0.0 or ranges[i] > scan_data.range_max:
        continue

    # Calculate laser point position in laser coordinate frame
    angle = scan_data.angle_min + scan_data.angle_increment * i
    point_in_laser = PointStamped()
    point_in_laser.header = scan_data.header
    point_in_laser.point.x = ranges[i] * math.cos(angle)
    point_in_laser.point.y = ranges[i] * math.sin(angle)
    point_in_laser.point.z = 0.0

    # Transform to base_link coordinate frame
    try:
        point_in_base = do_transform_point(point_in_laser, trans)
        x = point_in_base.point.x
        y = point_in_base.point.y

        # Determine obstacle position based on base_link coordinate
frame
        dist = math.sqrt(x**2 + y**2)
        if dist <= self.ResponseDist * 1.5:
            # Use atan2 to calculate angle relative to base_link x-axis
            angle_to_base = math.atan2(y, x) * RAD2DEG

            if abs(angle_to_base) < self.LaserAngle:
                self.front_warning += 1
            elif self.LaserAngle <= angle_to_base < 90.0:
                self.Left_warning += 1
            elif -90.0 < angle_to_base <= -self.LaserAngle:
                self.Right_warning += 1
        except TransformException:
            continue

    twist = Twist()
    current_time = self.get_clock().now().nanoseconds / 1e9

    # Determine current obstacle state
    front_blocked = self.front_warning > self.conut
    left_blocked = self.Left_warning > self.conut
    right_blocked = self.Right_warning > self.conut

    if not front_blocked:
        # Front is clear: side obstacles do not block forward motion,
        # just go straight and release the turn direction lock
        self.turn_dir = 0
        obstacle_state = 10
        twist.linear.x = self.linear
        twist.angular.z = 0.0
    else:
        # Front blocked: rotate in place. Pick a direction once and keep it

```

```

        # until the front is clear again, preventing left/right oscillation
        if self.turn_dir == 0:
            if left_blocked and not right_blocked:
                self.turn_dir = -1 # obstacle on left, turn right
            elif right_blocked and not left_blocked:
                self.turn_dir = 1  # obstacle on right, turn left
            else:
                self.turn_dir = 1 if self.Left_warning <= self.Right_warning
        else -1

        obstacle_state = 6 if self.turn_dir > 0 else 4
        twist.linear.x = 0.0
        twist.angular.z = self.turn_dir * self.angular

        # Log only when state changes or after log_interval
        if obstacle_state != self.last_obstacle_state or (current_time -
self.last_log_time) >= self.log_interval:
            log_messages = {
                4: 'Obstacle in front, turning right',
                6: 'Obstacle in front, turning left',
                10: 'Front clear, going forward'
            }
            if obstacle_state in log_messages:
                self.get_logger().info(log_messages[obstacle_state])
            self.last_log_time = current_time
            self.last_obstacle_state = obstacle_state

        self.pub_vel.publish(twist)

```

- Parameter file path

```
$HOME/microros_ws/src/microros_v2_bringup/config/common.yaml
```

Parameter descriptions and tuning suggestions for the `laser_Avoidance` node in `common.yaml`:

| Parameter                 | Default value | Description                                                                       | Tuning suggestions                                                                                                                           |
|---------------------------|---------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>linear</code>       | 0.15 m/s      | Linear speed while moving forward during avoidance                                | In narrow spaces it is recommended to lower to 0.1 m/s; in open areas it can be raised to 0.2 m/s                                            |
| <code>angular</code>      | 1.0 rad/s     | In-place turning angular velocity                                                 | If turning is too slow the response will be sluggish; it can be raised to 1.5 rad/s                                                          |
| <code>LaserAngle</code>   | 30.0°         | Half-angle of the front sector, determines the "straight ahead" judgment range    | The larger the angle, the more sensitive it is to lateral obstacles, but it may also cause unnecessary turns; recommended between 20°–45°    |
| <code>ResponseDist</code> | 0.3 m         | Obstacle response distance (the actual detection threshold is 1.5×, i.e., 0.45 m) | Too close risks collision; too far causes premature turns and longer detour paths; adjust according to the obstacle size in your environment |

```

laser_Avoidance:
  ros__parameters:
    linear: 0.15           # Forward linear speed (m/s)
    angular: 1.0           # In-place turning angular velocity (rad/s)
    LaserAngle: 30.0       # Half-angle of the front obstacle detection
    sector (deg)
    ResponseDist: 0.3       # Obstacle response distance (m), actual
    detection threshold is 1.5x this value

```